SE/COM S 3190 – Construction of User Interfaces

Final Project Technical Documentation

Fall 2025

Budget Buddy

Team Members

David Lawlor
dlawlor@iastate.edu

Jongwoo Kim
jwk0425@iastate.edu

Iowa State University
Department of Computer Science

Contents

1 Introduction

1.1 Project Overview

Budget Buddy is a comprehensive web-based personal finance management application designed to help users track their income, expenses, and budget goals efficiently. Built using the MERN stack (MongoDB, Express, React, Node.js), the application provides a user-friendly dashboard, real-time transaction logging, and visualized financial data.

Problem statement: Many students and young professionals struggle to track daily spending on mobile devices using spreadsheets or bank apps that lack custom categorization and quick logging. Budget Buddy addresses this by providing a responsive, simple CRUD interface with visualizations and budgeting tools.

Scope: Full-stack web application (responsive web app) with authentication, transaction CRUD, dashboard visualizations, and budget management.

1.2 Target Users and Use Cases

University student or young professional managing a limited monthly budget.

- **Needs:** record daily expenses quickly on mobile; understand spending breakdowns; set simple budgets.
- **Pain points:** spreadsheets are hard to view on phones; banking apps have limited custom categories and poor visualization.
- **Solution:** Budget Buddy offers a responsive UI, quick transaction logging, custom categories, and charts for insights.

Typical workflows / use cases: - Add a new transaction (food, transport) from the mobile view.
- Edit or delete a transaction from the transaction list.
- View monthly spending breakdown and budget progress on the dashboard.

Secondary users: Admin or support accounts (if implemented) to manage seeded data or moderate categories.

1.3 Core Features

- **Authentication:** JWT-based login/signup with token persisted to localStorage for session persistence.
- **Dashboard:** Total balance, recent transactions, interactive doughnut chart for expenses using react-chartjs-2.
- **Transactions CRUD:** Full Create, Read, Update, Delete on financial records via REST API endpoints.
- **Budgeting:** Users can input their monthly income and separate it into a budget

- **Unified Modal Form:** Single modal component for Add/Edit transactions to reduce navigation and provide a seamless experience.
- **Data backup:** Users can download or upload their data making for easy storage and safety.
- **Responsive Design:** Layout adapts to mobile with a hamburger menu and stacked dashboard components.

1.4 Originality vs Inspiration

The project idea is inspired by common personal finance apps and budgeting tools but adapted for quick mobile logging and classroom scope. Differences and improvements include the unified modal CRUD pattern, localStorage-based authentication persistence, and a student-focused responsive interface.

2 System Overview and Project Description

The application follows an MVC-style separation between frontend (React) and backend (Express/Node) with MongoDB as the database and Mongoose as the ODM.

2.1 Major Functional Modules

- **Authentication Module:** Login/Signup routes, JWT creation and verification, ProtectedRoute wrapper in frontend.

- **Dashboard Module:** Aggregates balance, recent transactions, and budget stats via `/api/dashboard/stats`. Uses Promise.all for concurrent fetches.

- **Transactions Module:** REST endpoints for transactions (`GET /api/transactions`, `POST /api/transactions`, `PUT /api/transactions/:id`, `DELETE /api/transactions/:id`).

- **UI Components Module:** React components for Dashboard, Transactions table, Modal form, Navigation, and Charts.

2.2 UI and Navigation Flow

- Landing / Login → Authenticated redirect to Dashboard.

- Dashboard shows summary + quick actions (Add Transaction modal).

- Transactions page shows a responsive table with Edit/Delete actions that open the Modal form in edit mode.

- Budget page displays budget lines with update/delete controls.

Conditional flows include redirecting unauthorized users to Login via `<ProtectedRoute>` and switching modal behavior based on `isEditing` state.

2.3 CRUD Entities

- **Transaction** (primary entity): - Fields: `date`, `description`, `amount`, `category`, `userId` (owner).
  - Actions: Create (add), Read (list), Update (edit via modal), Delete (confirm + delete).
  - Relationships: Transactions belong to Users (one-to-many).
- **User**: - Fields: `name`, `email`, `passwordHash`, `createdAt`.
  - Actions: Signup, Login, retrieve profile.
- **Budget (optional)**: - Fields: `category`, `limit`, `period`, `userId`.
  - Actions: Create, update, delete budget lines; compute progress relative to transactions.

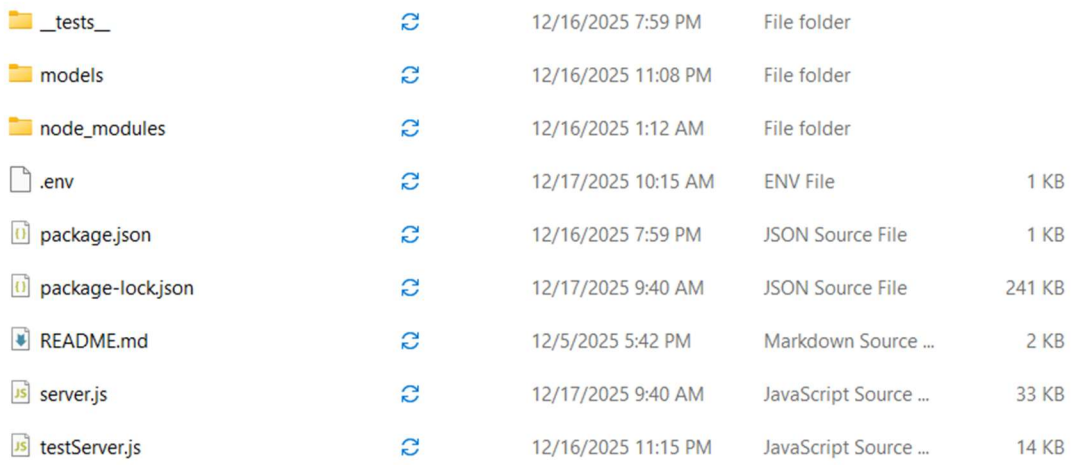
3 File and Folder Architecture

3.1 Folder Structure Overview

The project separates frontend and backend directories to improve maintainability. This allows independent development, deployment, and clearer separation of concerns.

3.2 Backend Folder Structure

3.2.1 Backend Folder Tree:



3.2.2 Backend Folder Description:

package.json

Defines the Node.js project metadata for the backend (nm_15_backend). It specifies the project name, version, and description ("Backend for Budget Buddy application"). This file lists all runtime dependencies such as Express, Mongoose, JSON Web Token (JWT), Bcrypt, and CORS, as well as development dependencies including Jest and Nodemon. It also defines scripts for starting the server, running the development server, and executing automated tests. The main entry point for the application is set to server.js.

server.js
Serves as the primary production server for the Budget Buddy backend. This file initializes the Express application, connects to MongoDB using Mongoose, and defines all RESTful API endpoints. Core functionality implemented in this file includes user authentication (signup and login using JWT), transaction management (create, update, delete, and search), budget management (create, update, delete), dashboard statistics calculation, recurring transaction handling, spending alerts, user profile management, and data backup and restore operations. It represents the central logic hub of the backend system.

testServer.js
A dedicated server implementation used exclusively for testing purposes. It mirrors the main API routes defined in server.js but uses mock data or simplified authentication in place of live database connections where appropriate. This separation allows unit and integration tests to be executed reliably without affecting production data.

README.md
Provides documentation for the backend system. It explains the purpose of the Budget Buddy backend, outlines implemented features (authentication, API endpoints, mock data usage), and includes setup and installation instructions. The README also lists available API endpoints and describes required environment variables.

.env
Stores environment-specific configuration values that should not be committed to version control. This file includes sensitive information such as the server port number (5001), the MongoDB connection URI, and the JWT secret key used for token signing and verification.

tests folder and api.test.js
Contains the automated test suite for the backend application. The api.test.js file uses Jest and Supertest to perform unit and integration tests on major API endpoints, including authentication (signup and login), transaction operations, budget management, and user profile functionality. These tests help ensure API correctness and reliability.

models folder
Contains all Mongoose schema definitions used by the application:

- **User.js:** Defines the User schema with personal details (first name, last name, email), authentication data (hashed password), user role (user or admin), and profile information such as bio, address, and preferences. Includes pre-save middleware for password hashing and instance methods for password comparison.

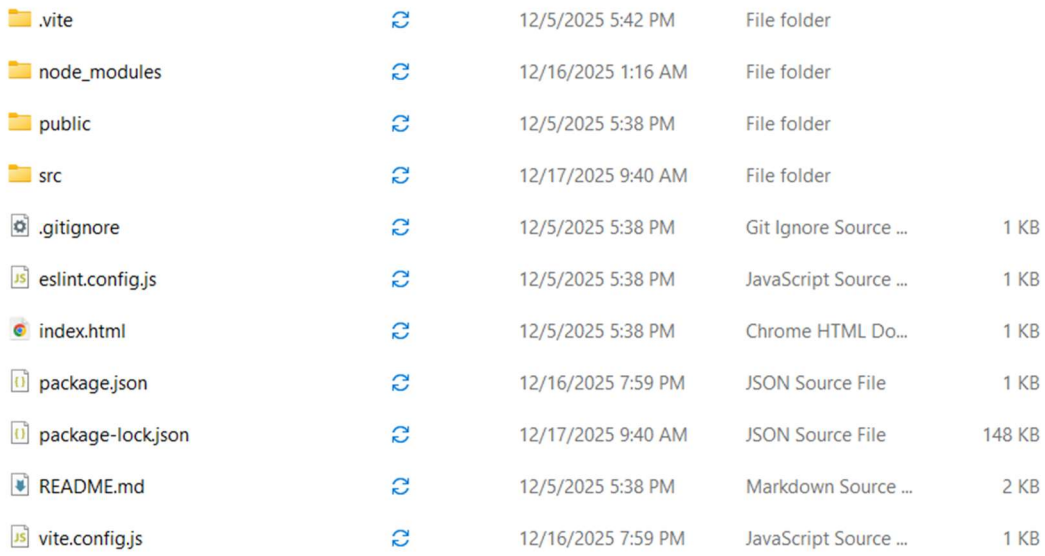
- **Transaction.js:** Defines the Transaction schema with fields for user reference, date, description, category, amount, transaction type (income or expense), and recurring transaction attributes such as recurrence pattern, next occurrence, and optional end date.
- **Budget.js:** Defines the Budget schema with user reference, category, budget amount, spent amount, remaining amount, and the budget period (monthly, weekly, or yearly).

package-lock.json
Automatically generated file that locks the exact versions of all installed dependencies and sub-dependencies. This ensures consistent and reproducible builds across different development and deployment environments.

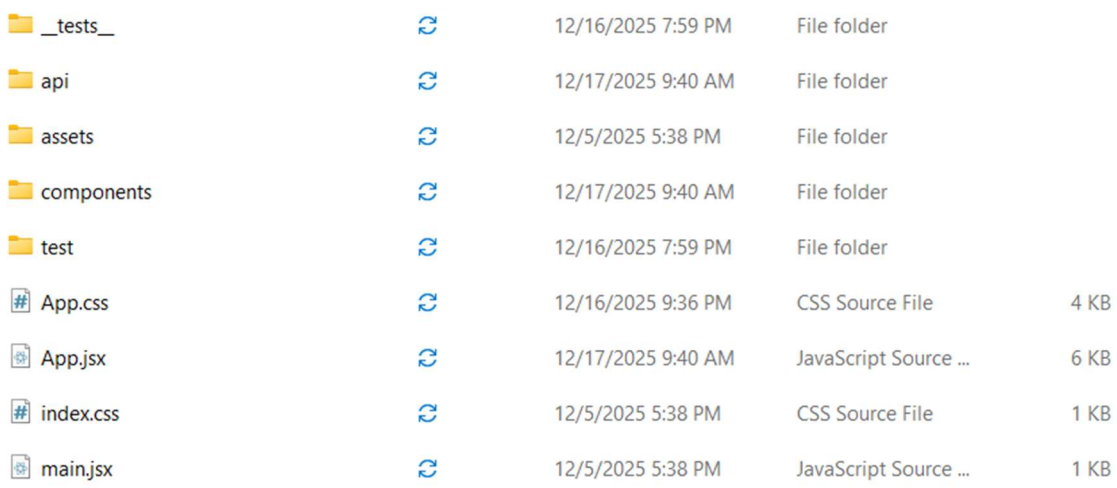
node_modules folder
Contains all installed Node.js dependencies required by the backend application, including Express, Mongoose, Bcrypt, JWT, CORS, and testing utilities. This folder is generated by npm and is typically excluded from version control.

3.3 Frontend Folder Structure

3.3.1 Frontend Folder Tree:

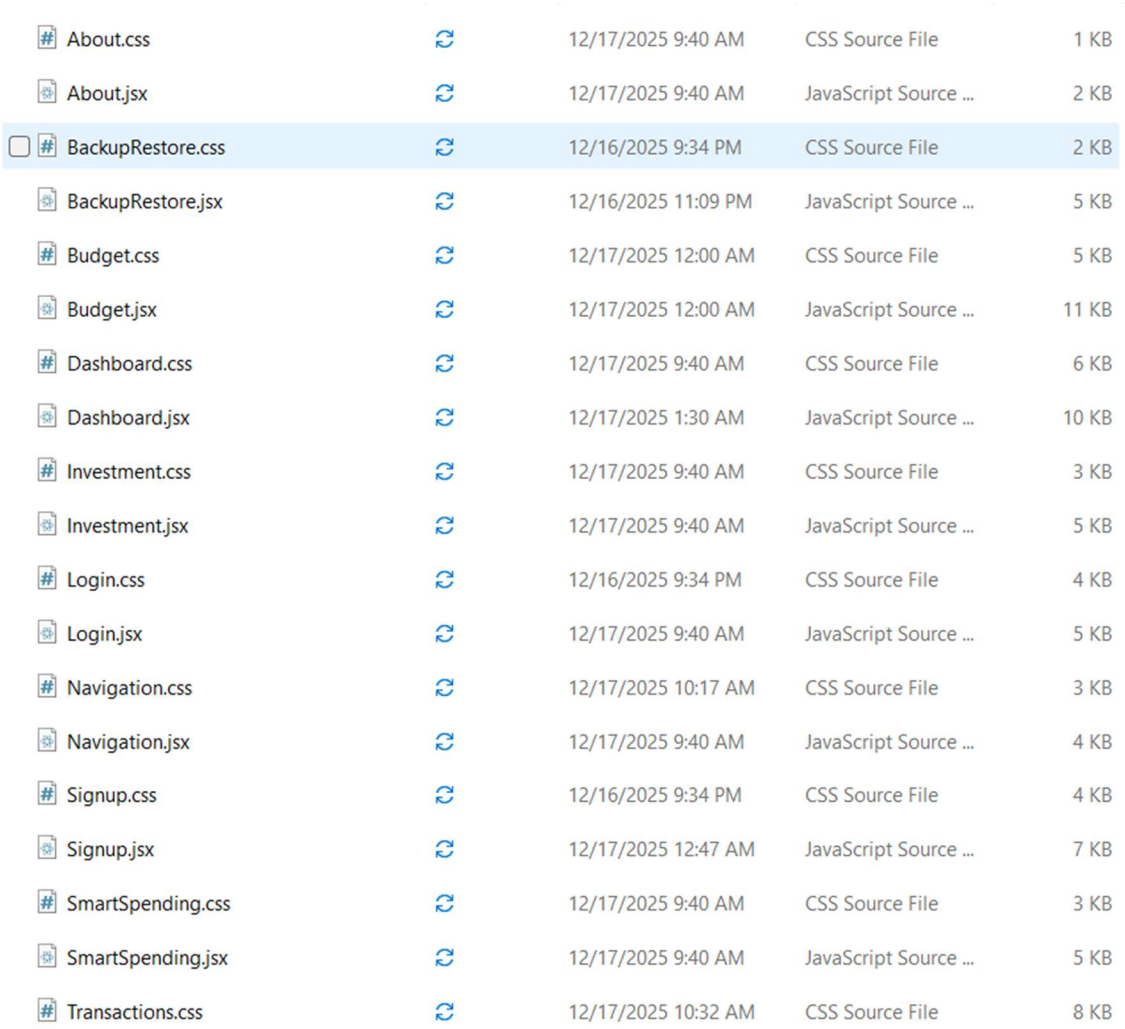


Inside src/



__tests__	12/16/2025 7:59 PM	File folder	
api	12/17/2025 9:40 AM	File folder	
assets	12/5/2025 5:38 PM	File folder	
components	12/17/2025 9:40 AM	File folder	
test	12/16/2025 7:59 PM	File folder	
App.css	12/16/2025 9:36 PM	CSS Source File	4 KB
App.jsx	12/17/2025 9:40 AM	JavaScript Source ...	6 KB
index.css	12/5/2025 5:38 PM	CSS Source File	1 KB
main.jsx	12/5/2025 5:38 PM	JavaScript Source ...	1 KB

Inside components/



About.css	12/17/2025 9:40 AM	CSS Source File	1 KB
About.jsx	12/17/2025 9:40 AM	JavaScript Source ...	2 KB
BackupRestore.css	12/16/2025 9:34 PM	CSS Source File	2 KB
BackupRestore.jsx	12/16/2025 11:09 PM	JavaScript Source ...	5 KB
Budget.css	12/17/2025 12:00 AM	CSS Source File	5 KB
Budget.jsx	12/17/2025 12:00 AM	JavaScript Source ...	11 KB
Dashboard.css	12/17/2025 9:40 AM	CSS Source File	6 KB
Dashboard.jsx	12/17/2025 1:30 AM	JavaScript Source ...	10 KB
Investment.css	12/17/2025 9:40 AM	CSS Source File	3 KB
Investment.jsx	12/17/2025 9:40 AM	JavaScript Source ...	5 KB
Login.css	12/16/2025 9:34 PM	CSS Source File	4 KB
Login.jsx	12/17/2025 9:40 AM	JavaScript Source ...	5 KB
Navigation.css	12/17/2025 10:17 AM	CSS Source File	3 KB
Navigation.jsx	12/17/2025 9:40 AM	JavaScript Source ...	4 KB
Signup.css	12/16/2025 9:34 PM	CSS Source File	4 KB
Signup.jsx	12/17/2025 12:47 AM	JavaScript Source ...	7 KB
SmartSpending.css	12/17/2025 9:40 AM	CSS Source File	3 KB
SmartSpending.jsx	12/17/2025 9:40 AM	JavaScript Source ...	5 KB
Transactions.css	12/17/2025 10:32 AM	CSS Source File	8 KB

- ### 3.3.2 Frontend Folder Description:
  package.json: Defines the React frontend project and its metadata. Lists core dependencies such as React (v19.2.0), React Router DOM, Chart.js, and development tools including Vite, Vitest, and ESLint.

vite.config.js: Vite configuration file that sets up the development server, enables the React plugin, and configures the testing environment using jsdom.

index.html: Main HTML entry point for the Vite-powered React application where the root React component is mounted.

eslint.config.js: Defines linting rules and coding standards to maintain code quality and consistency across the frontend codebase.

README.md: Documentation file describing the purpose, setup, and usage of the frontend application.

src/: Contains all source code for the React application, including components, API integration, assets, and tests.

src/components/: Houses React components for core pages and UI elements such as Login, Signup, Dashboard, Budget, and navigation.

src/api/: Contains files responsible for communicating with the backend API and handling HTTP requests.

src/assets/: Stores static assets such as images, icons, and other media used by the UI.

src/__tests__/: Includes unit and integration tests for React components and frontend logic.

public/: Contains static assets served directly by Vite, including the default vite.svg logo.

node_modules/: Stores all third-party libraries and dependencies installed via npm. This folder is auto-generated.

.vite/: Vite-generated cache and development server files used to speed up builds.

Overall Frontend Purpose: Implements the user interface for the Budget Buddy application, supporting transactions, budgeting, smart spending insights, user profiles, and backup/restore functionality.

4 Code Explanation and Logic Flow

This section explains how the application behaves internally, how data flows between components, and how requests are processed from the UI to the database.

4.1 Frontend–Backend Interaction

- Frontend uses `apiService.js` (fetch/axios) to call backend endpoints with JSON bodies and Authorization header bearing the JWT.

- On success, frontend updates component state via `useState` and re-renders charts/tables.

- Error handling uses `try/catch`, displays UI messages or toast notifications, and handles unauthorized responses by clearing localStorage and redirecting to Login.

4.2 React Component Example

```
const validateForm = () => {
  const newErrors = {};

  if (!formData.email.trim()) {
    newErrors.email = 'Email is required';
  } else if (!/\S+@\S+\.\S+/.test(formData.email)) {
    newErrors.email = 'Email is invalid';
  }

  if (!formData.password) {
    newErrors.password = 'Password is required';
  } else if (formData.password.length < 6) {
    newErrors.password = 'Password must be at least 6 characters';
  }

  return newErrors;
};
```

Login Component Example

Purpose within UI: Handles user authentication and redirects to dashboard after successful login

- Props received: Only receives setUser function to update parent component's user state
- Hooks used: useState (for form data, errors, loading state), useNavigate (for routing)
- Form handling: Includes validation for email format and password length, state updates for form fields, error handling
- Backend interaction: POST request to login endpoint using authAPI service
- Conditional UI: Loading states, error displays, conditional navigation

4.3 API Route Examples

POST /api/transactions

Creates a new transaction for the authenticated user.

• Validates required fields (date, description, category, amount, type)

• Converts amount to number and date to Date object

• Requires JWT authentication
• Saves a new Transaction document linked to the user
• Returns 201 on success, 401 if unauthorized, 500 on server error

GET /api/transactions

Retrieves all transactions for the authenticated user.
• Requires JWT authentication
• Queries transactions by user ID and populates user info
• Returns 200 on success, 401 if unauthorized, 500 on server error

PUT /api/transactions/:id

Updates an existing transaction owned by the authenticated user.
• Validates transaction ID and ownership
• Validates and converts updated fields
• Requires JWT authentication
• Updates and saves the transaction document
• Returns 200 on success, 404 if not found/unauthorized, 500 on server error

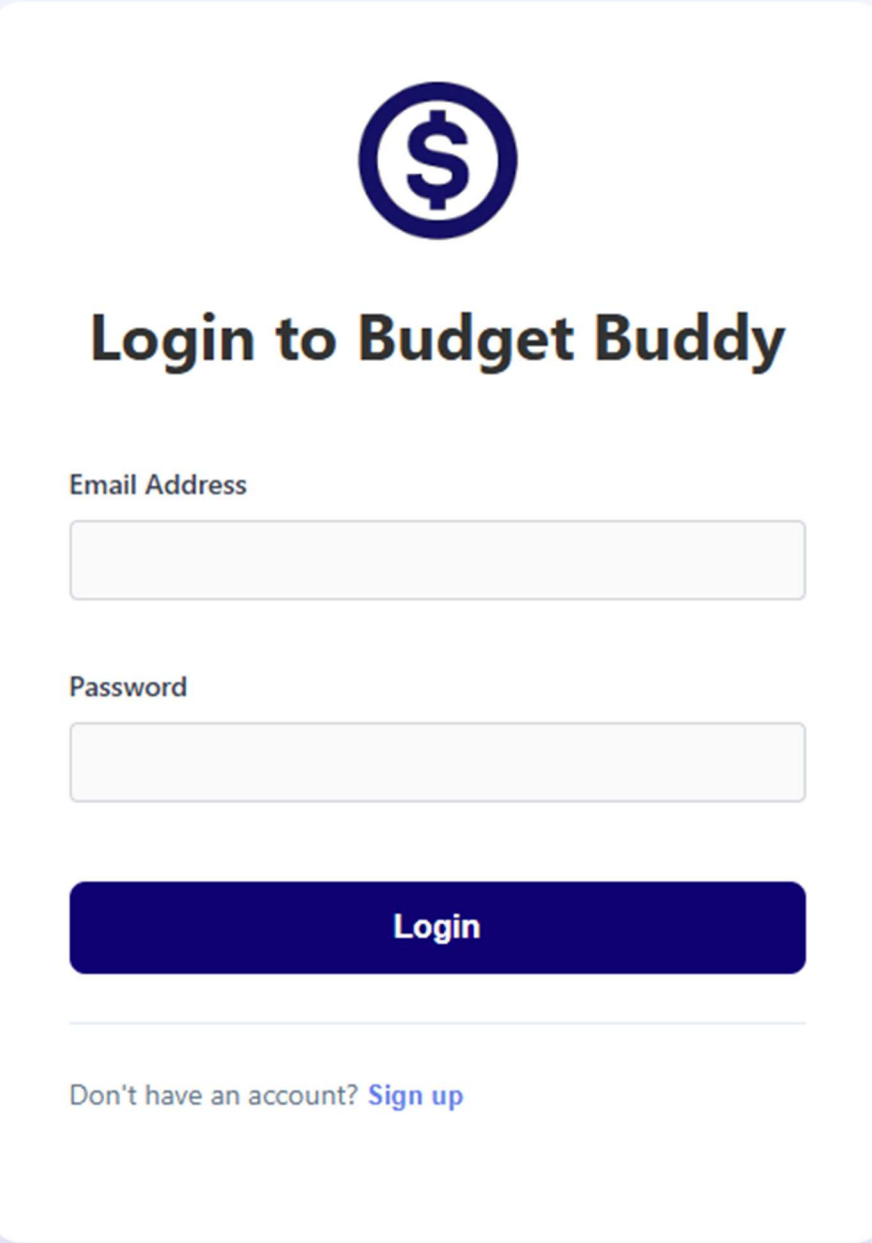
DELETE /api/transactions/:id

Deletes a transaction owned by the authenticated user.
• Validates transaction ID and ownership
• Requires JWT authentication
• Deletes the transaction from the database
• Returns 200 on success, 404 if not found/unauthorized, 500 on server error

4.4 Database Interaction

Budget Buddy uses MongoDB with Mongoose to define three main collections—User, Transaction, and Budget—each with schema-level validation, structure, and relationships. The **User** schema stores authentication and profile data, including name, email, hashed password, role (user or admin), active status, and an optional embedded profile (bio, avatar, address, preferences). It includes password-hashing middleware and timestamps. The **Transaction** schema stores financial records tied to a user through a userId reference, with fields for date, description, category, amount, and type (income or expense). It also supports recurring transactions via fields like isRecurring, recurrencePattern, nextOccurrence, and endDate, and includes timestamps. The **Budget** schema defines per-user spending limits by category, storing userId, category, budget amount, spent amount, remaining amount, and a period (monthly, weekly, yearly), with minimum numeric constraints and timestamps. Relationships are implemented using ObjectId references, forming one-to-many links from User → Transactions and User → Budgets. Validation is enforced through required fields, enums, and numeric constraints. Common queries include find() for user-specific data, findByIdAndUpdate() for updates, deleteOne() for removals, and populate() for joining related user data. Indexing considerations include MongoDB's automatic _id index, plus recommended indexes on userId and frequently filtered date fields to improve query performance.

5 Screenshots and Web Views



This is the login page where users can input valid login information to access the website. This using password hashing to store / pass passwords securely.

Budget Buddy

Create your account to start managing your finances

First Name

Last Name

Email Address

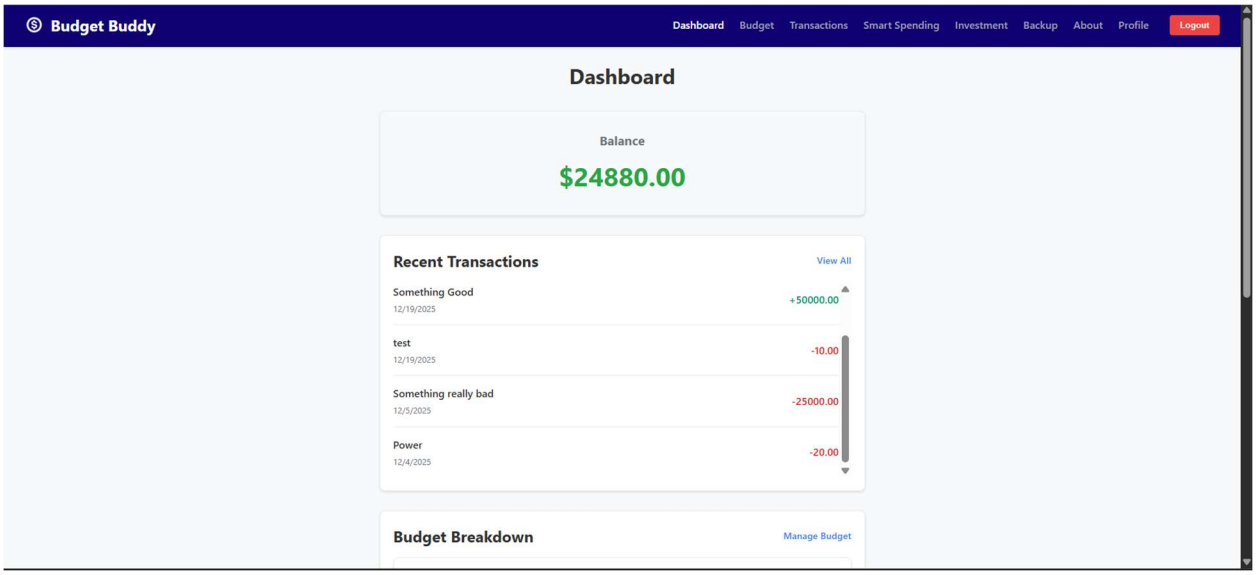
Password

Confirm Password

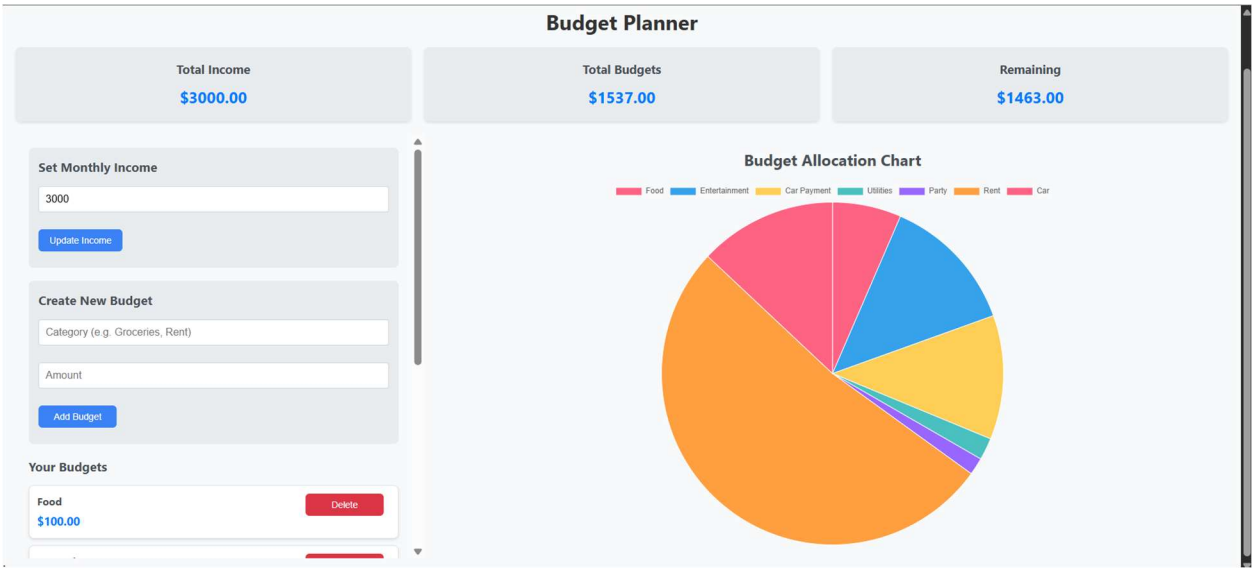
Create Account

Already have an account? Log in

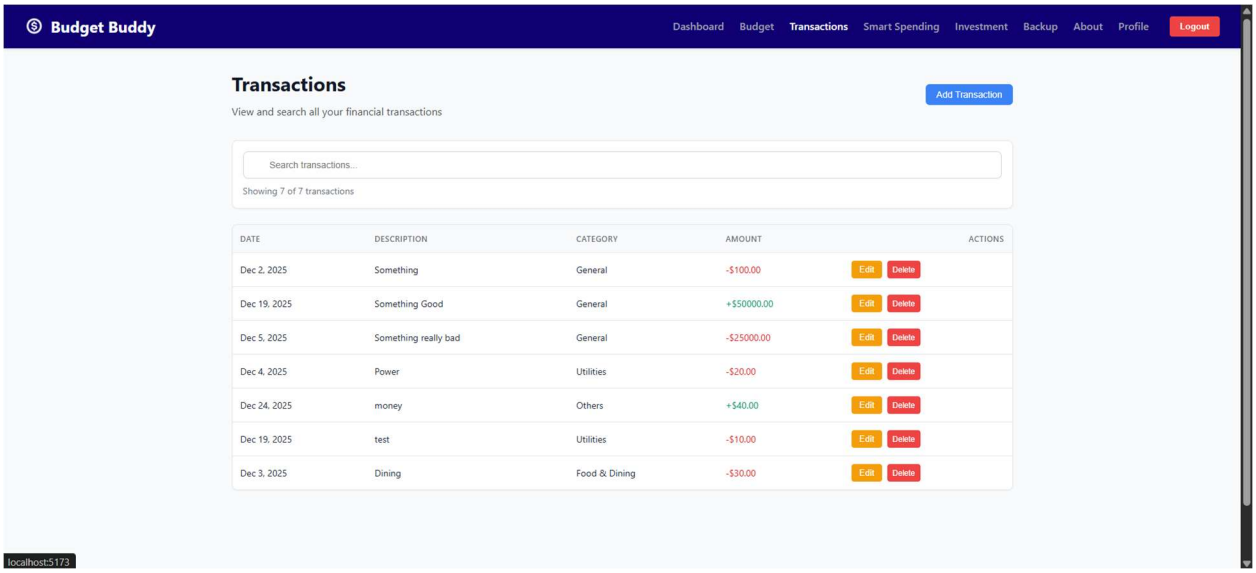
This is the sign-up page where users can create an account as long as it follows the valid parameters.



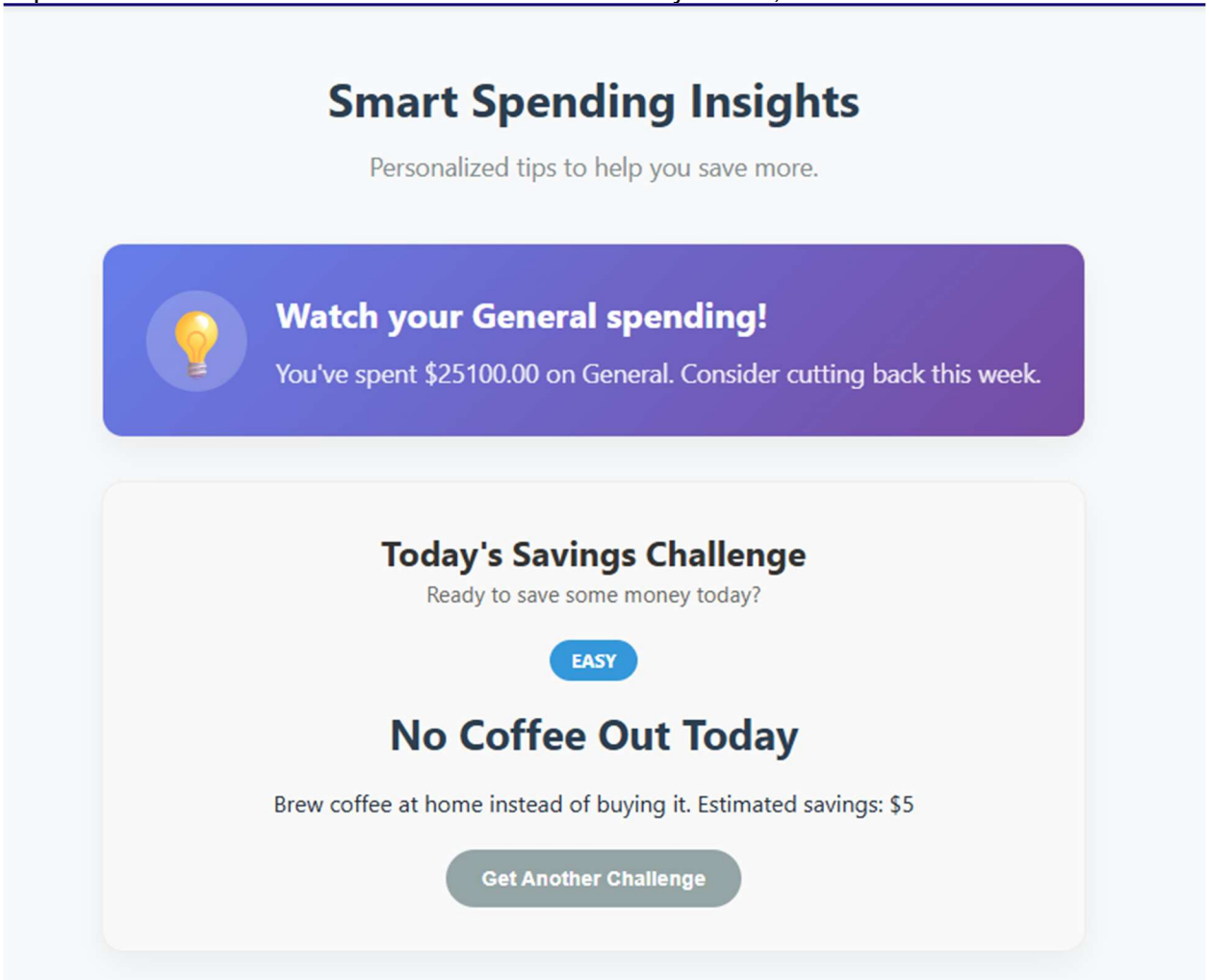
This is the dashboard where users first see after logging in or signing up. It serves as a summary providing users with ease of access to the info they need immediately.



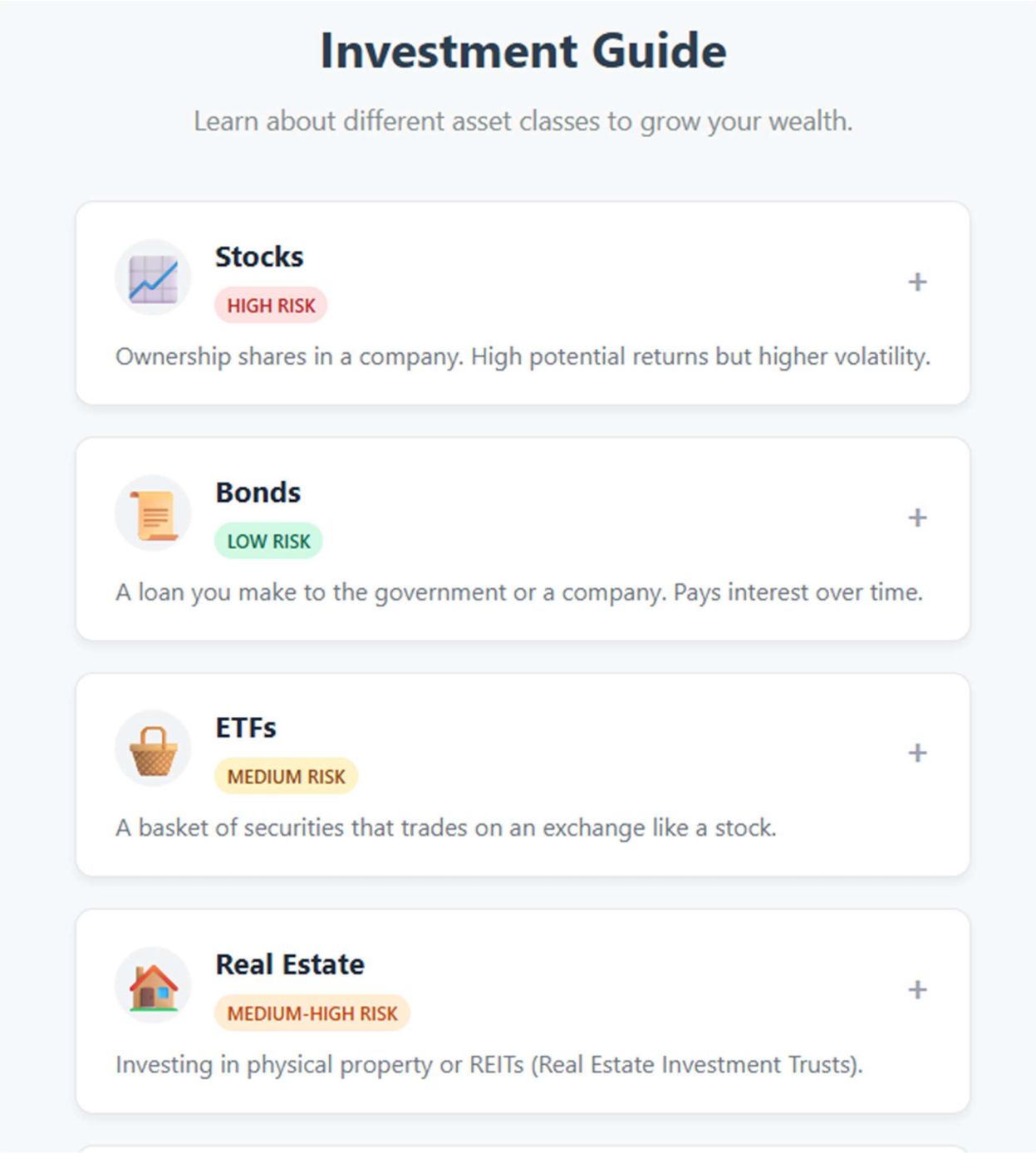
This is the budget creator. It stores info about the user in mongo db and is saved after every edit. It allows the user to create a monthly income and then split that income into different budgeting categories.



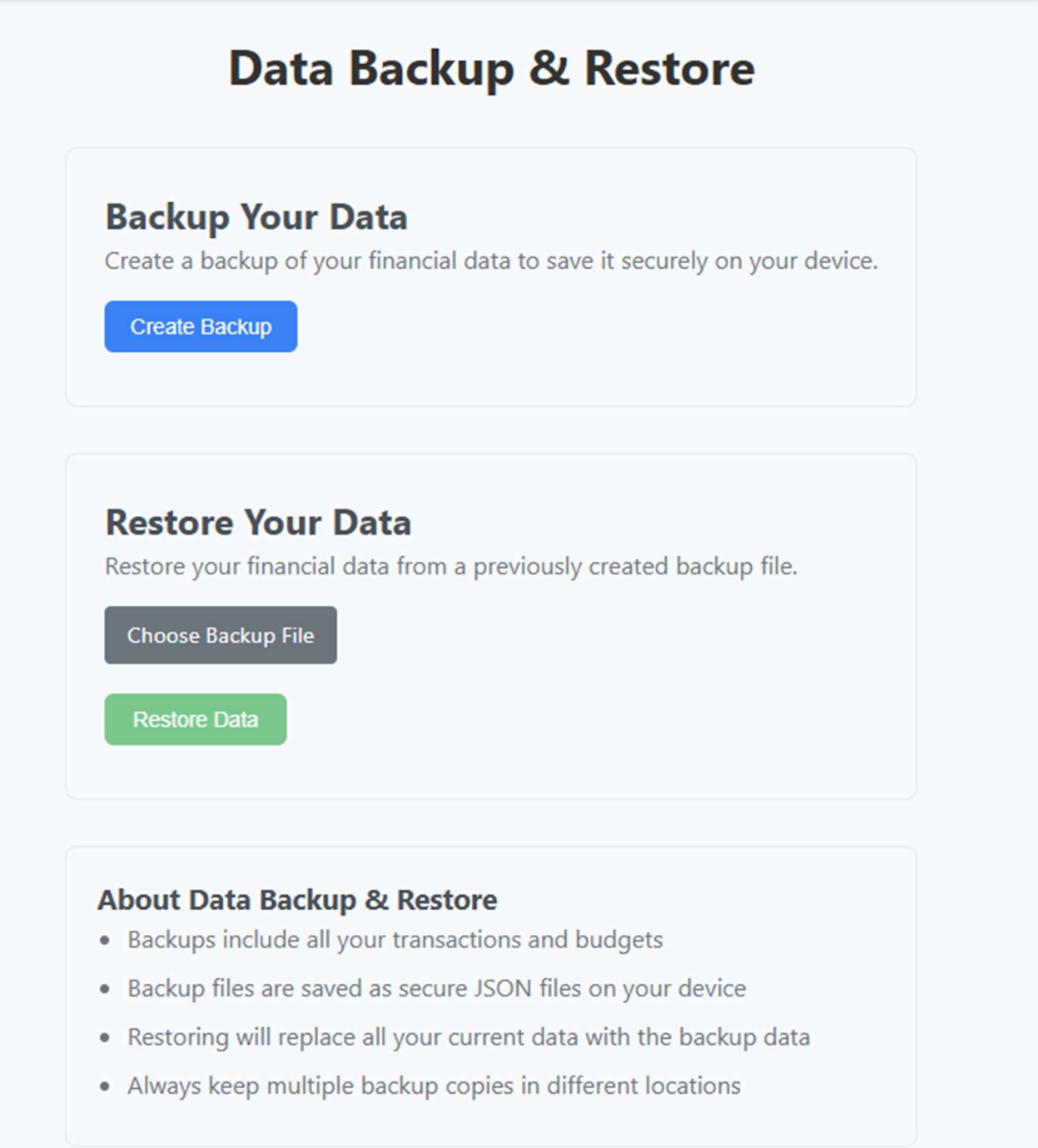
This is the transactions page. It allows the user to have a lot of all expenses and income they generate. It also auto fills the user's balance. The user will be able to export this as well as import transactions. Also transactions can be manually added, edited and deleted.



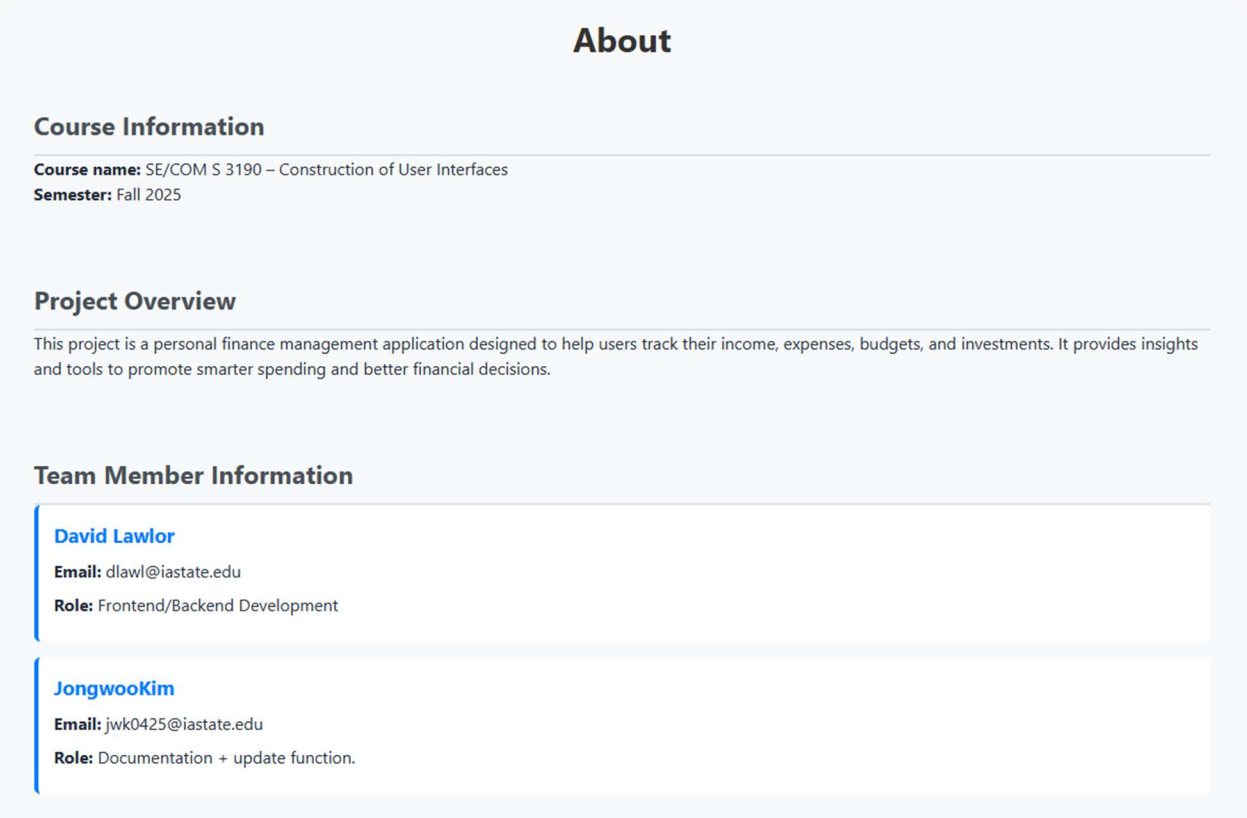
This is the smart spending tips page where users can receive tips for how to save or spend money.



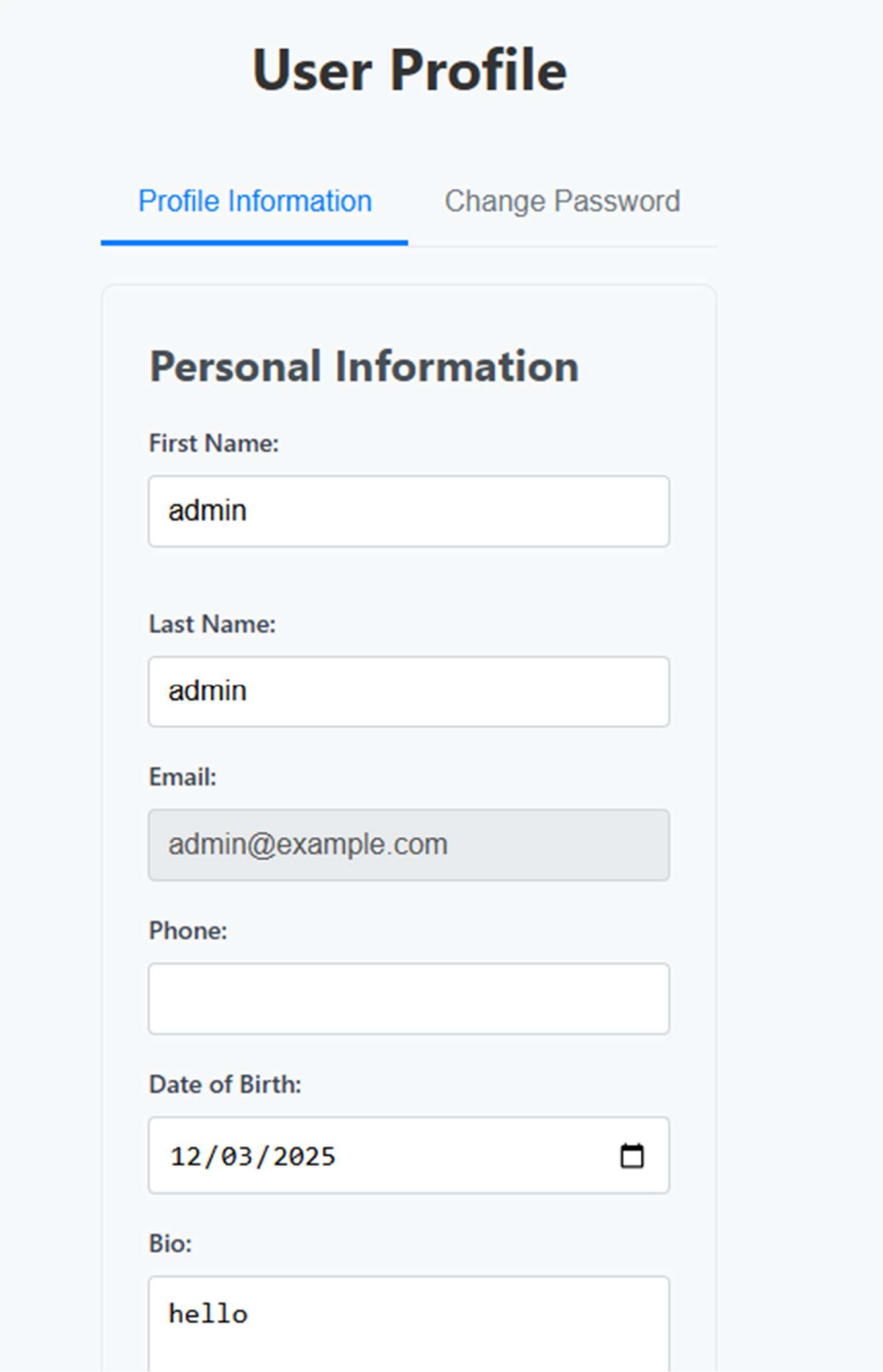
This is the investment Guide giving users insight on how to invest their money.



This is how users restore or backup their data. This is a good resource for importing transactions as mentioned before.



This is the about page giving users' information about the website and it's authors.



This is the profile page where users can edit information associated with their profile. They can change their profile and set many different profile preferences.

6 Installation and Setup Instructions

This section ensures your project can be successfully run by another developer.

6.1 Frontend Setup

1. `cd frontend` (or `cd NM_15_Front_End` if repo contains this folder)
2. `npm install`
3. `npm run dev`
4. Visit `http://localhost:5173`

6.2 Backend Setup

Give clear setup steps:

- `cd backend`
- `npm install`
- Create `.env` with the following variables (example):

```
PORT=5001
MONGODB_URI=mongodb://localhost:27017/budgetbuddy
JWT_SECRET=your_secret_key_here
```

```
Than npm start
```

6.3 Database Setup

- Use local MongoDB (compass).
- Create database `budgetbuddy` and ensure the connection string matches `.env`.
- Seed data: optionally provide a `seed.js` or route to insert sample users/transactions.

6.4 Environment Variables

- `PORT` — backend server port
- `MONGODB_URI` — MongoDB connection string
- `JWT_SECRET` — secret for signing JWTs
- `VITE_API_URL` or similar — frontend environment variable for API base URL

7 Contribution Overview

Each member must describe their exact contributions. Avoid vague phrases like “helped with backend” be explicit and technical.

7.1 Jongwoo Kim

- UI/UX design and responsive navigation
- Transaction CRUD implementation
- Smart Spending features (dashboard logic)

7.2 David Lawlor

Frontend and Backend Development for the following
- Login/Signup pages
- Transactions
- Budget Page
- Profile Page
- Data Backup Page

8 Challenges Faced

8.1 David Challenges

- **Problem:** React state reset upon refresh causing forced re-login.
- **Diagnosis:** State initialization did not check localStorage on mount.
- **Solution:** Initialize auth state from `localStorage` and use `useEffect` to sync.
- **Lessons learned:** Persist critical session data and handle token expiry.

8.2 Jongwoo Challenges

- **Problem:** 404 when editing transactions.
- **Diagnosis:** Missing PUT route on backend and mismatched frontend endpoint.
- **Solution:** Added `app.put('/api/transactions/:id')` and synchronized `apiService.js` to call PUT with auth header.
- **Lessons learned:** Ensure backend routes and frontend calls match; integration tests help catch these early.

9 Final Reflections

Each team member reflects individually.

9.1 David Reflection

As the backend lead for Budget Buddy, I gained substantial experience designing and implementing a RESTful API using Node.js, Express, and MongoDB with Mongoose. I improved my understanding of authentication workflows by implementing JWT-based login and signup functionality, as well as securing routes with middleware. Debugging issues related to API routing, database queries, and frontend integration helped me become more methodical in isolating and resolving backend problems. If given more time, I would enhance the system with more advanced analytics queries, improved indexing, and stronger error handling. This project reinforced the importance of clean schema design, thorough testing, and clear separation of concerns in full-stack development.

9.2 Jongwoo Reflection

Working on the Budget Buddy frontend allowed me to significantly improve my skills in React, particularly with component-based design, state management, and client-side routing using React Router. One of the most rewarding aspects of the project was designing a responsive and intuitive user interface that made complex financial data easy to understand through dashboards and charts. Implementing transaction CRUD functionality and integrating visualizations using Chart.js helped deepen my understanding of frontend–backend communication. If redesigning the project, I would focus on further improving UI consistency and accessibility, as well as refactoring components for greater reusability. Overall, this project strengthened my confidence in building scalable React applications and translating user needs into functional interfaces.